

Reviewer Pack — Minimal Local Ring Norm Correlation with Circuit Monotone Wi...

Entry b22e25230e66 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 14:45:43 UTC

Minimal Local Ring Norm Correlation with Circuit Monotone Width via Tropicalization

Entry ID: b22e25230e66 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 14:45:43 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuits

Statement:

The minimal local ring norm of the tropical dual of a Boolean circuit φ is linearly correlated with its monotone width $w(\varphi)$, such that the minimal local ring norm $|t_norm(\varphi^*)| = \Theta(w(\varphi))$ for a constant $c > 0$, where $t_norm(\varphi^*)$ is the minimal local ring norm of the tropical dual of φ .

Rationale (proposer's reasoning):

Tropical geometry provides a framework to study discrete objects in a continuous setting, which may reveal deeper structural properties. If the local ring norm of the tropical dual of circuits can be related to their monotone width, it could provide insights into circuit complexity and its computational hardness.

Taxonomy category: TROPICALGEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f6d429a3b21923a5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated circuits φ , the correlation coefficient between $|t_norm(\varphi^*)|$ and $w(\varphi)$ across at least 25 of 30 random seeds is greater than or equal to 0.8, with no seed producing a metric value greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Boolean circuits" AND minimal local ring norm"
- "tropicalization" AND monotone width AND circuit" - "minimal local ring norm" correlation
with tropical duality in Boolean circuits

Top relevant hits considered: - [http://arxiv.org/abs/1901.01180v2] Decomposing tropical rational functions - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/2401.00252v2] Cluster algebras and monotone Lagrangian tori - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

# Helper functions for Gaussian elimination and matrix operations
def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i + A[i:].index(max(abs(row[i]) for row in A[i:]))
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def min_local_ring_norm(tropical_circuit):
    # Placeholder implementation, should be replaced with actual computation
```

```

    return random.random()

def monotone_width(circuit):
    # Placeholder implementation, should be replaced with actual computation
    return random.randint(1, 10)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    correlations = []

    for n in n_values:
        instances_tested = 0
        total_t_norm = 0
        total_width = 0

        while instances_tested < 30:
            circuit = [random.choice([0, 1]) for _ in range(n)]
            t_norm = min_local_ring_norm(circuit)
            width = monotone_width(circuit)

            if t_norm > 0 and width > 0:
                total_t_norm += t_norm
                total_width += width
                instances_tested += 1

        if instances_tested == 0:
            return {
                "metric_name": "correlation",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "No valid instances found"
            }

        avg_t_norm = total_t_norm / instances_tested
        avg_width = total_width / instances_tested
        correlation = avg_t_norm * avg_width / (avg_t_norm**2 + avg_width**2)
        correlations.append(correlation)

    mean_corr = sum(correlations) / len(correlations)
    std_corr = math.sqrt(sum((x - mean_corr)**2 for x in correlations) / len(correlations))

    return {
        "metric_name": "correlation",
        "metric_value": mean_corr,
        "instances_tested": 30 * len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": mean_corr >= 0.8 and std_corr <= 10,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, **result}}")
    results.append(result)

mean_corr = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
std_corr = math.sqrt(sum((r["metric_value"] - mean_corr)**2 for r in results if r["metric_value"] is not None) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Not supported\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=No valid instances found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 11, **result}
TRIAL: {"seed": 23, **result}
TRIAL: {"seed": 37, **result}
TRIAL: {"seed": 53, **result}
TRIAL: {"seed": 71, **result}
TRIAL: {"seed": 89, **result}
TRIAL: {"seed": 103, **result}
TRIAL: {"seed": 127, **result}
TRIAL: {"seed": 149, **result}
TRIAL: {"seed": 167, **result}
TRIAL: {"seed": 191, **result}
TRIAL: {"seed": 211, **result}
TRIAL: {"seed": 233, **result}
TRIAL: {"seed": 257, **result}
TRIAL: {"seed": 277, **result}
TRIAL: {"seed": 311, **result}
TRIAL: {"seed": 347, **result}
TRIAL: {"seed": 389, **result}
TRIAL: {"seed": 421, **result}
TRIAL: {"seed": 463, **result}
TRIAL: {"seed": 503, **result}
TRIAL: {"seed": 547, **result}
TRIAL: {"seed": 593, **result}
TRIAL: {"seed": 631, **result}
TRIAL: {"seed": 677, **result}
TRIAL: {"seed": 727, **result}
TRIAL: {"seed": 773, **result}
TRIAL: {"seed": 821, **result}
TRIAL: {"seed": 877, **result}
TRIAL: {"seed": 929, **result}

```

RESULT: FALSIFIED counterexample="Not supported" first_failing_seed=11

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder implementations for both the minimal local ring norm and monotone width calculations, returning random values. This does not provide a meaningful empirical test of the conjecture as it does not compute the actual metrics required for the correlation.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code uses placeholder implementations for both the minimal local ring norm and monotone width calculations, returning random values. This doe | next: Implement correct algorithms for calculating the minimal local ring norm and monotone width to perform an accurate empirical test of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13747
2	propose	ollama_remote	glm4:latest	0	0	12984
3	preregistration	ollama_remote	glm4:latest	0	0	14644
4	novelty	ollama_remote	glm4:latest	0	0	8549
5	novelty	ollama_remote	glm4:latest	0	0	11760
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	94149
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19468
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11888
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20099
10	critic	ollama_remote	glm4:latest	0	0	10687
11	judge	ollama_remote	glm4:latest	0	0	9380

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 227354 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b22e25230e66.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b22e25230e66.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b22e25230e66.tar.gz (if generated)